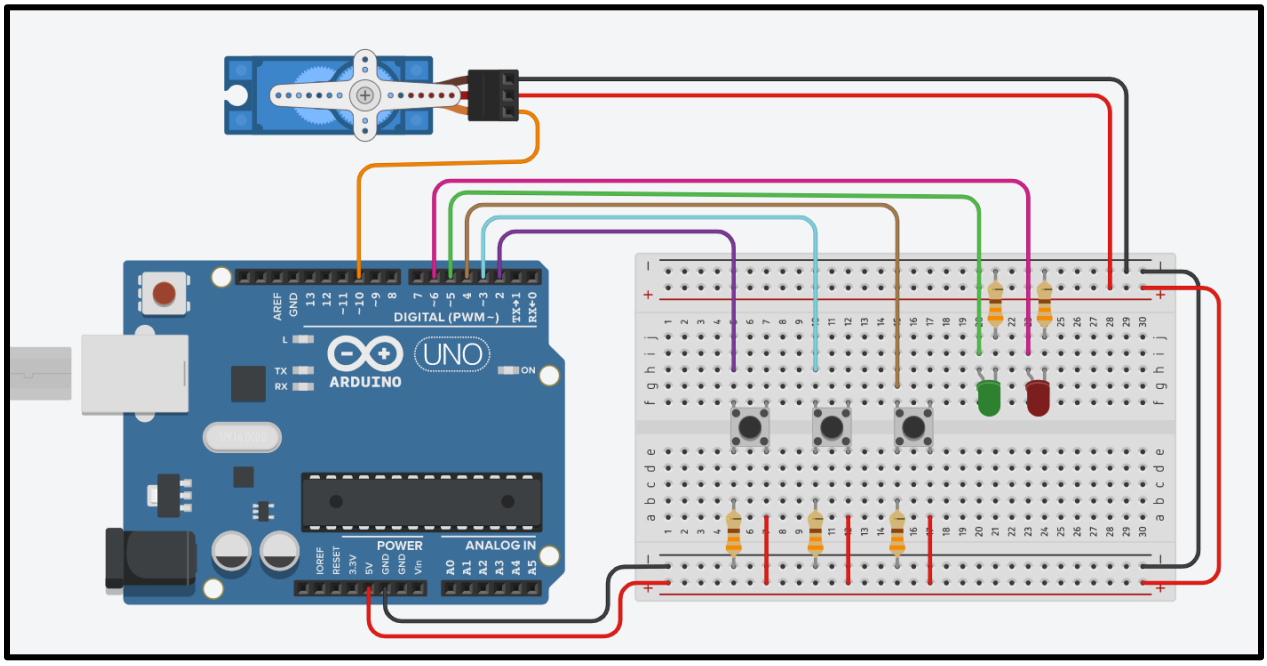
Motion – 04 – Combination Safe

Purpose: Learn how to create a circuit that uses a combination of button presses to move a servo motor(s) to unlock a box. You can add lights and a sound to enhance your lock box.

Materials:
2 LEDs, 5 - 220 or 330 Ω resistors, 15 jumper wires, 1 servo motor, 3 buttons

Procedure:

1. Wire the buttons, servo, and leds to the Arduino using the following schematic. Please notice that the top and bottom grounding and positive rails on the breadboard are joined using jumper wires.

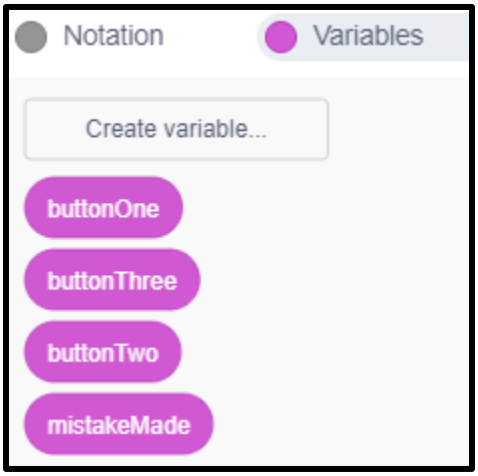


2. The Code. We'll use TinkerCAD to simulate and code this project using the **block code** interface. Once this is complete, we can copy the **text code** paste it into the Arduino IDE to upload to the Arduino.

Variables.

For this code we will need **variables**. Variables are used to store information to be referenced and manipulated in a computer program. They also provide a way of labeling data with a descriptive name, so our programs can be understood more clearly by the reader and ourselves. It is helpful to think of variables as containers that hold information. Their sole

purpose is to label and store data in memory. This data can then be used throughout your program.
Let's assign a variable to each button so that we can call on them throughout our program, and we'll add a fourth variable to keep track if a mistake was made or not:



To do this, we need to go to the Variables palette and press the **create variable** button. Enter the four variable names as above.

Note: *Variable names can't have spaces in them! TinkerCAD won't stop you from doing this, but your program will not run if you make this mistake.*

We have wired our red light to pin 6, our green light to pin 5, our servo that is going to lock the box to pin 10. Therefore we need to set the following starting values using our blocks.



With these values, our red light will be on, green light will be off, the box will be locked.

Initialize Variables. While there are many ways we could write the logic for this project, one limitation we have while coding with blocks only in TinkerCAD is no access to creating functions

of our own. The approach we will take, then, will be that *one time through the loop()* function will be a *full attempt to unlock the lock.* To accomplish this, we'll be using a **while loop.**

Before adding that, let's first add some initialization code so that each time loop() wraps around that our button variables get reset back to 0:



mistakeMade will track whether a user has made a mistake yet. 0 will stand for "no mistake" and 1 will stand for "a mistake was made". So the rest of our code will be wrapped in a loop to repeat over and over until the three buttons have been pressed:



Inside this **while loop**, we will want to check for button presses. When one is detected, we'll then need to decide if it was a correct press or a mistake press.

For now, let's build the program with a correct sequence of:

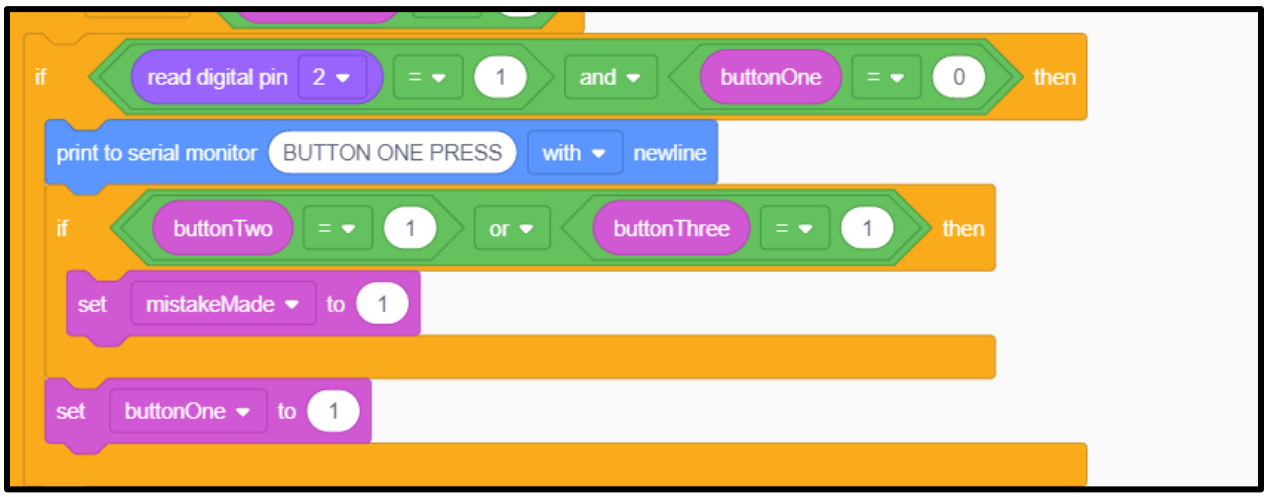
BUTTON 1 *(pin 2)* → BUTTON 2 *(pin 3)* → BUTTON 3 *(pin 4)*

Security is low, I guess! One challenge later on will be to modify what the correct order is.



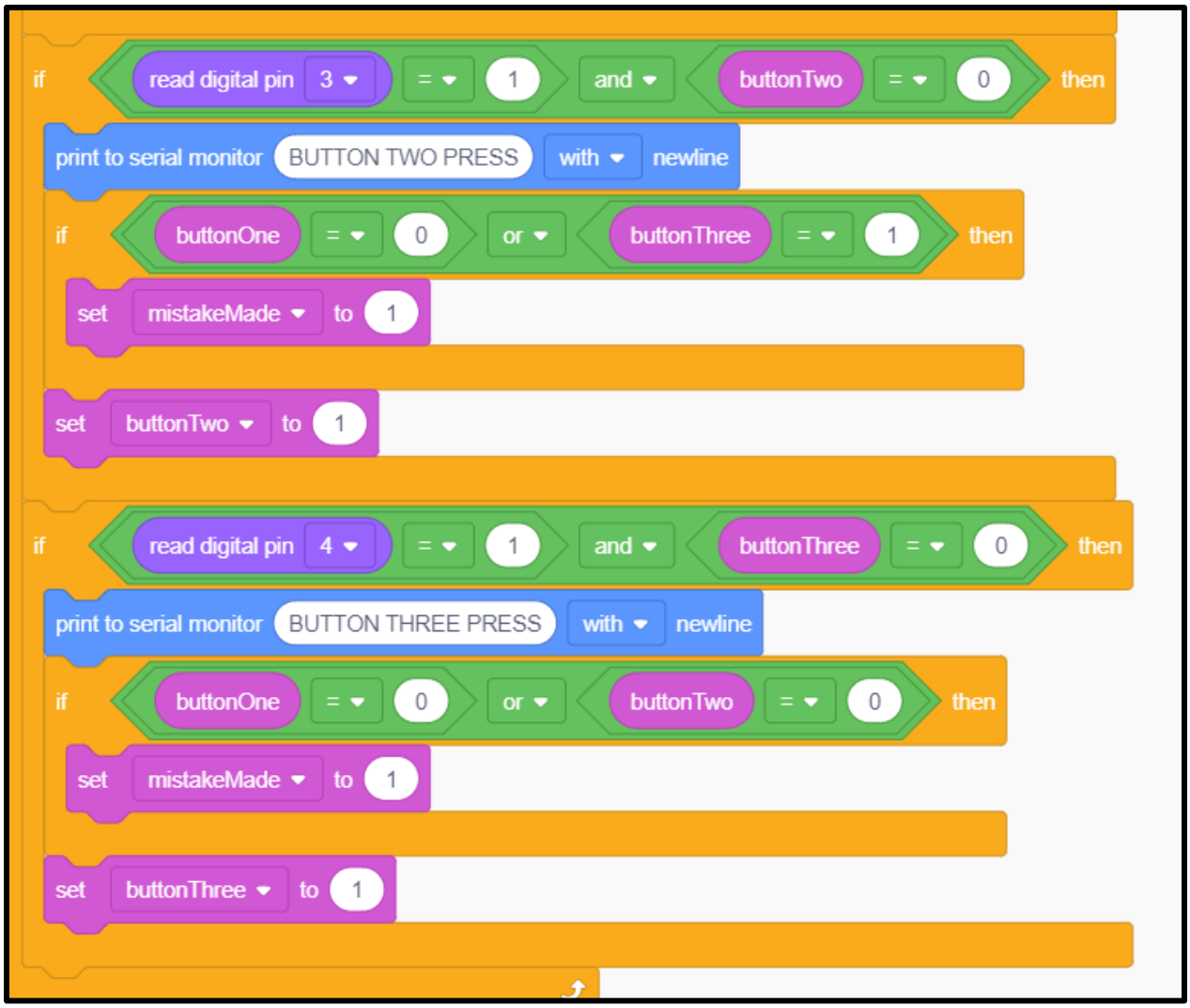
This type of conditional says “If Button 1 is being Pressed, **and** it hasn’t been pressed yet”. Notice how the second part of that condition is implicit because of what happens if the condition is true. **buttonOne** gets set to 1, which will stand for “Button One” was pressed. **Note:** *This is a great time to print a message to the Serial Monitor that we’ve pressed button one. We can use this when we test to verify this section happens one time, the first time Button One is pressed.*

The second part of what needs to go inside the **if statement** is to check if this button was entered in the right order. Since our order is B1 → B2 → B3, we can tell a mistake was made if **buttonTwo** or **buttonThree** variables still are holding a 1 (implying that the button has been pressed already). If this isn’t the case, let’s change **mistakeMade** to indicate an out-of-order press.

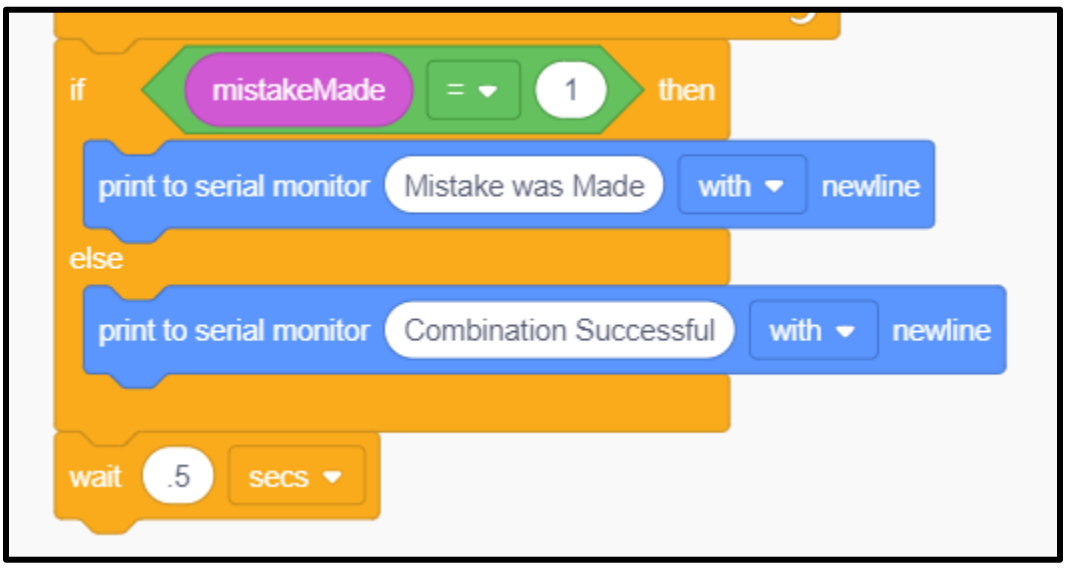


Remember that if **mistakeMade** gets set to 1 this while loop will end the next time it wraps around (which will seem pretty much instantaneously), at which point we can have some final output before letting the program start again.

Before that, though, let’s repeat the logic above to handle the other two button presses:

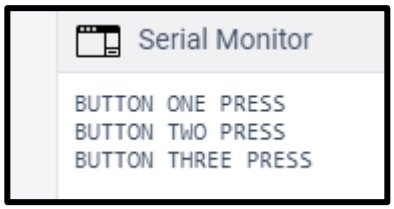


So the mechanism above should cause the **while loop** to stop as soon as a mistake was made. At that point, we'll want to active some LED and possibly move the servo. However, before that let's test the result in a simpler way with the **Serial Monitor.** Add the following script *after the while loop:*



If you run the program and view the Serial Monitor as you press the buttons, you should see that pressing an out-of-order button does indeed show the loop breaking and the message of a mistake being reported correctly:





However, if the correct order is entered, we don't get the success message. This is because our while loop's **loop condition** only has to do with the **mistakeMade** variable. We need to expand the condition to also stop the loop when a correct combination is made!

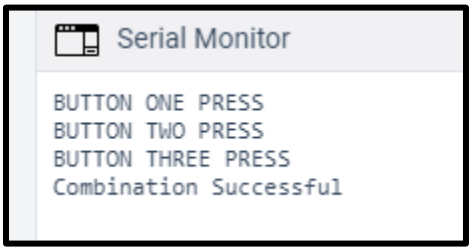
One way to do this would be to also break the loop if all three Button variables are holding a 1, since this implies that all three have been pressed. So let's tell the loop to only continue as long as *at least one of the buttons still holds a 0:*



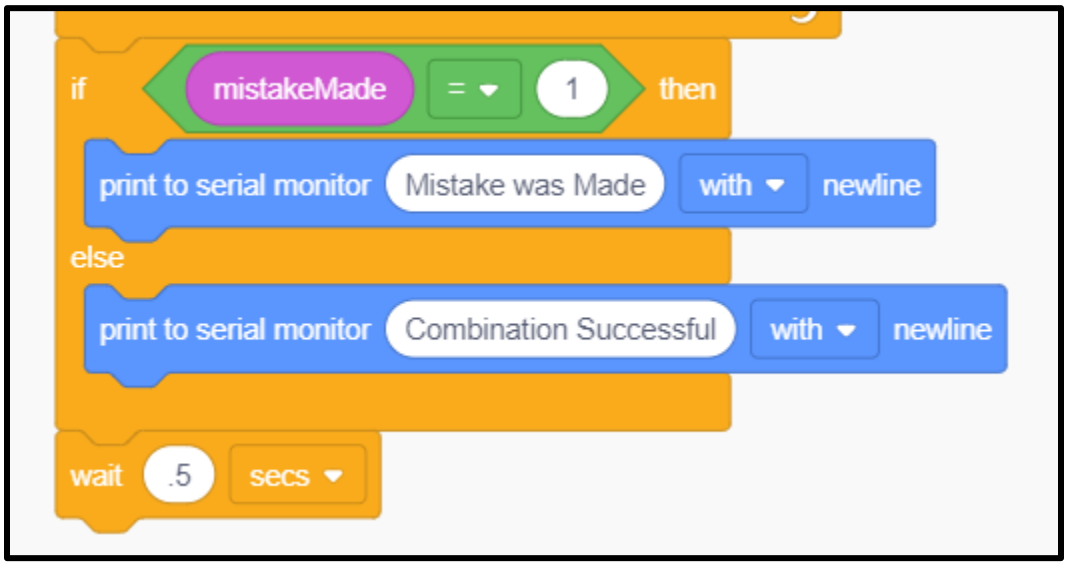
Note carefully how these are nested. If we aren't careful, we could change the meaning of the condition. When the condition above is combined with our original one, the result is this:



That's a pretty involved block! When articulating similar things using **text syntax**, it is thankfully a little more concise. If we try our test run again at this point, a correct combination in the **Serial Monitor** should yield:



Lastly, we should activate the appropriate LED and possibly activate the Servo Arm depending on the contents of **mistakeMade** when the loop is complete. You'll want to add your code to the **if/else** statement we created earlier:



Implement the following actions **on your own:**

If a Mistake was made:

- Flash the Red LED on and off 5 times (100ms on, 100ms off). Use a **repeat 5** loop to do this in a concise way

If the Correct Combination was entered:

- Turn the Red LED off.
- Turn the Green LED on.
- Rotate the Servo to 170 degrees. Keep it here for 7 seconds
- During the last 2 seconds, flash the green LED on and off 4 times (250ms on, 250ms off) to indicate that the look will be closing again soon.
- Rotate the Servo back to its initial position of 10 degrees.

Extension Challenges:

Successfully completing the project up until this point will earn a base grade of 8 out of 10. To earn the remaining 2/10 marks, complete at least four of the following extension challenges (each will be worth 0.5 marks).

1. Change the combination required to open the servo. You will need to adjust the logic statements inside the **while loop** to do this.

2. Incorporate a **piezo buzzer** into the circuit. Have it play one of two things when the while loop ends:
   - Some buzz tone or pattern when a mistake was made
   - A short melody when the correct combination was entered

3. Add three additional LEDs, one to represent each of the three buttons. Whenever a button is pressed, the corresponding LED should light up.

4. Expand the system so that it would work for a “lock box” that uses two servos working in unison. Add a third yellow LED as well.
   - if a correct combination is entered, change the servo movement slightly:
     - rotate the first servo to 170 (unlock mechanism)
     - rotate the second servo to 90 (default should also be 10 – door mechansim)
     - modify the servo close timing: at 7 seconds begin closing the door.
       -while the door is closing, turn the green LED off and the yellow LED on
       -Once the door is closed, “lock the door” and turn the yellow LED off.

   A suggested wiring diagram *(for extension four)* is as follows. However, if you think you can wire it without the reference, please do so!

